IAM

HANDWRITTEN WORK NOTES

AWS IAM

Users, groups, roles, policies, temporary credentials, federation, workload identities, guardrails and more

Learn the big picture
Who calls AWS, how requests are authenticated, and how authorization is evaluated.

Use it at work
Practical AWS examples, cross-account access, CI/CD identity, troubleshooting and operating checks.

Remember faster
Mind maps, formulas, diagrams, comparison tables and plain-language definitions.

Build safely
Temporary credentials, least privilege, MFA, guardrails, Access Analyzer and audit guidance.

```
PRINCIPAL -> REQUEST -> POLICY EVALUATION
DEFAULT DENY | EXPLICIT ALLOW | EXPLICIT DENY WINS
```

STS

Study edition: August 2026

What is AWS IAM?

The security control system for identities and permissions inside AWS accounts.

Simple definition: AWS Identity and Access Management (IAM) controls **who or what can call AWS**, **which actions they can request**, **which resources they can target**, and **under what conditions**.

Office security analogy

Identity register

IAM stores users, groups, roles, policies and identity-provider connections for an AWS account.

ID card check

Credentials prove the caller: password, access key, MFA, SAML/OIDC token or temporary role credentials.

Door access list

Policies decide whether a requested API action is allowed on the named AWS resource.

Security guardrails

Boundaries, SCPs, RCPs, session policies and explicit denies reduce the maximum possible access.

Main use cases

- Give administrators and engineers controlled access to AWS accounts.
- Allow EC2, Lambda, ECS and other workloads to call AWS services without hardcoded keys.
- Provide cross-account access by assuming roles.
- Federate workforce users from Microsoft Entra ID, Okta or another identity provider.
- Restrict access by resource, Region, network path, MFA, tags or organization.
- Audit who performed an API action and which role session was used.

Key point

IAM does not contain your EC2 data, S3 objects or database rows. It contains identities and policies used to access AWS services and resources.

The request journey

Principal — Person, workload or service → **Authenticate** — Credentials prove identity → **Request** — Action + resource + context → **Evaluate** — All applicable policies → **Decision** — Allow or deny

Memory sentence: IAM identifies the caller, AWS builds the request context, policies are evaluated, and the service allows or denies the API operation.

Where IAM fits in AWS

AWS Organizations, AWS accounts, IAM and IAM Identity Center solve different parts of access management.

AWS ORGANIZATION → ORGANIZATIONAL UNITS → AWS ACCOUNTS → RESOURCES

ORG

AWS Organization

Groups multiple AWS accounts, applies organization-wide guardrails and supports centralized governance.

ACCT

AWS account

A major security, billing and resource boundary. Each account has one root user and its own IAM resources.

RES

AWS resources

S3 buckets, EC2 instances, databases, queues, functions, keys and other service resources.

Do not mix up these four

Concept	What it is	Typical purpose
AWS account root user	The original account identity with powerful account-level access.	Only exceptional root-only tasks and recovery.
AWS IAM	Account-level identities, roles, credentials and permission policies.	Authorize AWS API requests inside or across accounts.
IAM Identity Center	Central workforce access service for multiple accounts and applications.	SSO, users/groups, permission sets and temporary role sessions.
AWS Organizations	Multi-account hierarchy and governance service.	Accounts, OUs, SCPs, RCPs and consolidated administration.

Single-account starter

Secure the root user, create a controlled administrative access path, and prefer roles with temporary credentials.

Enterprise pattern

Organization + many accounts + IAM Identity Center + external IdP + permission sets + organization guardrails.

Boundary shortcut

Organization	Govern many accounts together.
Account	Own resources and isolate blast radius.
IAM role	Carries temporary permissions for a session.
Policy	Expresses allowed or denied API actions.

> **Design principle:** Use multiple accounts for strong isolation; use IAM policies for fine-grained access inside and between those accounts.

AWS IAM master mind map

Use this page as the navigation map for the rest of the notes.



1. Human identities

Root user, IAM users, IAM Identity Center users, groups, federation and MFA.

2. Workload identities

IAM roles, service roles, service-linked roles, instance profiles and web identity roles.

3. Credentials

Passwords, access keys, MFA devices, SAML/OIDC assertions and STS temporary credentials.

4. Permissions

Identity policies, resource policies, managed policies, inline policies and conditions.

5. Guardrails

Permissions boundaries, session policies, SCPs, RCPs and VPC endpoint policies.

6. Federation

IAM Identity Center, SAML 2.0, OIDC, external IdPs, workforce access and CI/CD.

7. Policy evaluation

Implicit deny, explicit allow, explicit deny, union, intersection and cross-account rules.

8. Attribute access

Principal tags, resource tags, request tags, session tags and ABAC conditions.

9. Audit and analysis

CloudTrail, credential reports, Access Analyzer, last-accessed data and policy simulator.

Core formula

```
WHO / WHAT  +  ACTION  +  RESOURCE  +  CONDITIONS
                          ↓
ALL APPLICABLE POLICIES  →  ALLOW OR DENY
```

Authentication: Can AWS verify the caller and credential?

Authorization: Do the evaluated policies permit this exact request?

Learning order: Account and root → principals → credentials → roles → policies → evaluation → federation → workloads → governance → operations.

Core IAM vocabulary

These terms appear in policies, error messages, CloudTrail events and architecture diagrams.

Identity	An IAM user, user group or role that can have permission policies. A group helps assign permissions but does not make requests itself.
Entity	An IAM user or IAM role that can be authenticated and authorized. Groups are not IAM entities.
Principal	The actual caller making a request: root user, IAM user, role session, federated identity, AWS service or other supported principal.
Resource	The AWS object being targeted, such as an S3 object, KMS key, DynamoDB table, secret or IAM role.
Action	The API operation being requested, such as `s3:GetObject`, `ec2:StartInstances` or `kms:Decrypt`.
Request context	Facts AWS uses during evaluation: caller, action, resource, time, IP, VPC endpoint, tags, MFA, organization, session and more.
ARN	Amazon Resource Name - the standard identifier used to name AWS resources and principals in policies.

One request, broken into parts

```
Caller:   arn:aws:sts::123456789012:assumed-role/ReportAppRole/session-42
Action:   s3:GetObject
Resource: arn:aws:s3:::company-reports/2026/august.csv
Context:  Region, source IP/VPC endpoint, tags, organization, session data...
```

Identity vs principal

A role is an identity. After someone assumes it, the resulting **role session** is the principal making requests.

Group warning

An IAM group grants permissions to its users, but cannot be placed in the `Principal` element of a resource policy.

Friendly name vs ARN

Console names are human-friendly; ARNs identify the exact account, service, resource type and resource.

Common ARN shapes

Resource	Example shape
IAM role	`arn:aws:iam::123456789012:role/AppRole`
STS role session	`arn:aws:sts::123456789012:assumed-role/AppRole/session`
S3 bucket	`arn:aws:s3:::company-reports`
S3 objects	`arn:aws:s3:::company-reports/*`

Memory shortcut: Principal performs an Action on a Resource while AWS checks the Request Context.

Authentication and credentials

Credentials prove identity; policies still decide what the authenticated principal may do.

PWD

Console password

Long-term sign-in credential for root or an IAM user.

KEY

Access key

Access key ID + secret access key for signed API, CLI and SDK requests.

MFA

MFA

Additional factor that strengthens sign-in and can be required by policy.

STS

Temporary credentials

Access key ID + secret + session token + expiration.

Long-term versus temporary

Area	Long-term credentials	Temporary role credentials
Common source	Root or IAM user password/access key	AWS STS after role or federation
Lifetime	Remain valid until changed, disabled or deleted	Expire automatically
Extra value	Access key ID + secret	Access key ID + secret + session token
Preferred use	Only where a supported temporary method is not practical	Human federation, workloads, cross-account and CI/CD
Main risk	Leaked credentials remain usable for longer	Shorter blast window; permissions still must be correct

How API authentication works

CLI / SDK
Finds credentials

→

Signs request
AWS Signature Version 4

→

AWS verifies
Key, token, time and signature

→

IAM evaluates
Authorization policies

Never: place access keys in source code, AMIs, container images, tickets, email or shared scripts.

Prefer: IAM roles, Identity Center, OIDC federation, instance/task roles and automatic SDK credential refresh.

Authentication success does not equal authorization success. A valid credential can still receive `AccessDenied` because the requested action is not allowed.

Root user and human access strategy

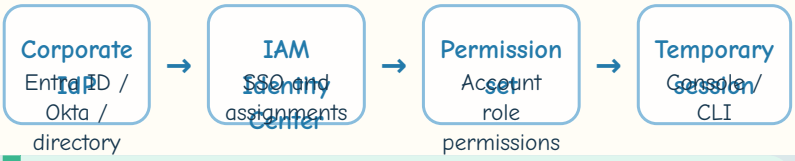
Protect the account root and give people temporary, traceable access for everyday work.

AWS account root user

Created with the AWS account and capable of highly privileged account operations.

- Use only for tasks that truly require root.
- Enable resilient MFA protection.
- Do not create root access keys.
- Protect recovery email and phone channels.
- Monitor every root sign-in and API event.

Preferred workforce path



Best default: workforce users authenticate through federation and receive temporary role credentials instead of individual IAM user access keys.

Root security checklist

- [ ] Use a strong, unique root password.
- [ ] Register more than one protected MFA method where your recovery design supports it.
- [ ] Store root access procedure in a controlled break-glass process.
- [ ] Keep the account email inbox and phone number current.
- [ ] Do not use root for routine administration.
- [ ] Alert on root sign-in and root API activity.
- [ ] For Organizations, evaluate centralized root access for member accounts.
- [ ] Test recovery and approval procedures without exposing credentials.

Break-glass access

Break-glass identity means a tightly controlled emergency path used only when normal federation is unavailable. Protect it with MFA, minimal standing permissions, approvals, logging, alarms and a documented post-use review.

When might an IAM user still exist?

Legacy integration · Specific unsupported workload · Emergency access design · Service-specific credential requirement

Decision rule: Start with federation or a role. Create a long-term IAM user only after documenting why temporary credentials cannot meet the use case.

IAM users and user groups

Useful concepts to understand, but not the preferred workforce model for a modern multi-account environment.

IAM user

- Identity created inside one AWS account.
- Can have a console password, access keys and MFA.
- Can receive policies directly or through groups.
- Has long-term credentials unless you make it assume a role.
- Does not automatically exist in other accounts.

IAM user group

- Collection of IAM users for permission assignment.
- Policies attached to the group flow to all members.
- A user can belong to multiple groups.
- Groups cannot be nested.
- A group is not an authenticated principal.

Group permission pattern



Important: The users make requests using their own credentials. The group only contributes permissions.

User versus group

Question	IAM user	IAM group
Can authenticate?	**Yes**	No
Can have policies?	Yes	Yes
Can appear as Principal?	Yes, where supported	No
Main purpose	Represent one account-local identity	Share permissions across users

Lifecycle operating checks

Joiner

Create only when justified, add correct groups, require MFA and avoid unnecessary keys.

Mover

Remove old groups and direct policies before granting the new job access.

Leaver

Disable console/access keys immediately, review activity, then remove the user safely.

Common mistake: Creating one IAM user per person in every AWS account. Use IAM Identity Center for centrally managed workforce access across accounts.

IAM roles: the central AWS identity pattern

A role carries permissions but has no permanent password or access key of its own.

```
ROLE = TRUST POLICY (WHO MAY ASSUME) + PERMISSIONS POLICIES (WHAT THE SESSION MAY DO)
```

Trust policy

Resource-based policy attached to the role.

- Names trusted AWS accounts, roles, users, services or federated providers.
- Usually allows an STS assume-role action.
- Can restrict source identity, external ID, OIDC claims, MFA or organization.

Permissions policies

Identity-based policies attached to the role.

- Allow or deny AWS service actions.
- Scope actions to resources and conditions.
- Become active only during a role session.

Role assumption - both sides matter

Caller policy
Allows `sts:AssumeRole`

+

Role trust policy
Trusts the caller

→

STS role session
Temporary credentials

→

Role permissions
Actions allowed in session

Human role

Engineer assumes a read-only or administrator role for a limited session.

Service role

EC2, Lambda, ECS or another service assumes a role for your workload.

Cross-account role

A principal from Account A assumes a trusted role in Account B.

IAM user versus IAM role

Area	IAM user	IAM role
Permanent association	One account-local person/application identity	Assumable by trusted principals
Long-term credentials	Can have password and access keys	None attached to the role
Credentials used	Long-term or after assuming a role	Temporary STS credentials
Preferred modern use	Exceptional cases	Humans, workloads, federation and cross-account access

Memory sentence: Trust gets you into the role; permission policies decide what you can do after you enter it.

AWS STS and temporary sessions

AWS Security Token Service creates short-lived credentials for roles, federation and delegated access.

Access key ID

Public identifier for the temporary credential set.

Secret access key

Secret value used to sign requests.

Session token

Required proof that the keys belong to an STS session.

Every STS credential set also has an expiration time. After expiration, the caller must obtain a new session.

AssumeRole flow

1. **Caller** — User, role, IdP or workload
2. **STS request** — Role ARN + session details
3. **Trust check** — May this principal assume?
4. **Temp keys** — Key + secret + token
5. **AWS APIs** — Role permissions apply

Common STS operations

Operation	Typical caller	Use
`AssumeRole`	AWS principal	Role switching, delegation and cross-account access
`AssumeRoleWithSAML`	SAML-federated workforce identity	Enterprise federation into an IAM role
`AssumeRoleWithWebIdentity`	OIDC token holder	CI/CD, web identity and supported workload federation
`GetSessionToken`	IAM user	Temporary credentials, often with MFA

Session controls

Session name and source identity

Help identify the original person or process in logs. Use meaningful, traceable values.

Session tags and session policy

Carry attributes into the session and optionally reduce its permissions for that session.

Role chaining

Using one role session to assume another role is called role chaining. Its session duration is more restricted than a direct initial role assumption, so design long automation paths carefully.

```
aws sts get-caller-identity
# Confirms the active Account, Principal ARN and UserId before you run a change.
```

Operational habit: Run `aws sts get-caller-identity` before production work so you know exactly which account and role session are active.

IAM policy types - the big picture

Different policy types grant access, limit access or control a specific resource path.

Identity-based
Attached to a user, group or role. Describes what that identity may do.

Resource-based
Attached to a resource. Names the principals that may access that resource.

Permissions boundary
Maximum permissions an IAM user or role can receive from identity policies.

Session policy
Limits one STS or federated session. It does not create permissions by itself.

SCP / RCP
Organization guardrails that cap principal permissions or resource access.

Endpoint policy
Controls which actions can pass through a supported VPC endpoint.

The five questions inside a policy

Effect Allow / Deny + **Action** Which API? + **Resource** Which ARN? + **Principal** Who? Resource policies + **Condition** Only when?

Granting policies versus limiting policies

Policy type	Can grant permission?	Main job
Identity-based policy	**Yes**	Grant actions to an identity
Resource-based policy	**Yes**	Grant access to the resource and name principals
Permissions boundary	No	Set maximum identity permissions
Session policy	No	Reduce one temporary session
SCP / RCP	No	Set organization-level maximums

Critical: A guardrail that says an action is allowed does not give the caller that action. A granting policy must still allow it.

Specialized access controls

ACLs and RAM shares
Some services have legacy ACLs or use AWS Resource Access Manager for supported resource sharing.

KMS key policies
AWS KMS uses key policies as a central authorization control; identity permission alone may not be sufficient.

Memory shortcut: Identity policy = what this identity can do. Resource policy = who may access this resource. Guardrail = maximum possible access.

Anatomy of an IAM JSON policy

Read policies statement by statement: Effect, Action, Resource and optional Condition.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListOnlyReportsPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::company-reports",
      "Condition": {
        "StringLike": { "s3:prefix": "reports/*" }
      }
    },
    {
      "Sid": "ReadReportObjects",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::company-reports/reports/*"
    }
  ]
}
```

Version — Policy language version, normally `2012-10-17`. It is not the policy revision number.

Statement — One or more permission decisions inside the document.

Sid — Optional human-friendly statement identifier.

Effect — `Allow` or `Deny`.

Action — API operations, using the service namespace.

Resource — ARN or ARNs affected by the statement.

Condition — Additional rules evaluated against request context.

Principal — Used in resource-based and trust policies, not ordinary identity-based policies.

Why two S3 statements?

Bucket action

`s3:ListBucket` targets the **bucket ARN**. The prefix is limited using a condition.

Object action

`s3:GetObject` targets the **object ARN pattern** ending in `/reports/*`.

Least-privilege writing checklist

- [ ] Start from the exact business task and required API calls.
- [ ] Replace broad actions such as `s3:*` with specific actions.
- [ ] Replace `Resource: "*"` where the service supports resource scoping.
- [ ] Add conditions for network, organization, tags, MFA or other valid context.
- [ ] Validate the JSON and review IAM Access Analyzer findings.
- [ ] Test the intended allow and at least one intended deny.

> **Tip:** Use the Service Authorization Reference to confirm which actions support resource-level ARNs and which condition keys are valid.

How AWS evaluates a request

The safest mental model is: start denied, find applicable allows, then let any explicit deny win.

```
1. DEFAULT = IMPLICIT DENY
2. AN APPLICABLE EXPLICIT ALLOW IS REQUIRED
3. ANY APPLICABLE EXPLICIT DENY OVERRIDES THE ALLOW
```

Simplified evaluation path

Authenticate — Who is the principal? → **Build context** — Action, resource, tags, IP… → **Collect policies** — Identity, resource, boundary, org… → **Check denies** — Explicit deny? → **Decision** — Allow or deny

Union and intersection

Union - sources that can grant

Within the same account, identity-based and resource-based permissions can contribute allows.

`IDENTITY ALLOW ∪ RESOURCE ALLOW`

Intersection - maximum controls

Boundaries, session policies and organization guardrails reduce what remains effective.

`GRANTS ∩ BOUNDARY ∩ SESSION ∩ SCP/RCP`

Decision examples

Identity policy	Boundary / guardrail	Explicit deny	Result
Allows `s3:GetObject`	Allows it	None	**Allowed**
Allows `s3:GetObject`	Does not allow it	None	**Denied**
No applicable allow	Allows all	None	**Denied** - guardrail grants nothing
Allows action	Allows action	Applicable deny	**Denied**

Cross-account reminder

For most cross-account requests, the caller side must allow the request and the target account must trust or grant access. A failure on either side denies the request.

Troubleshooting rule: Do not inspect only the policy attached to the caller. Check resource policy, trust policy, boundary, session policy, SCP, RCP, endpoint policy and KMS controls too.

Managed, inline and boundary policies

Choose reusable policy objects for normal permissions and boundaries for delegated administration.

Type	Managed by	Reusable?	Best use
AWS managed policy	AWS	Yes, across many principals	Quick start and common job/service permissions; review broad access.
Customer managed policy	Your account	Yes	Preferred reusable least-privilege permissions under your control.
Inline policy	Your account	No - embedded in one identity	Strict one-to-one permission that should not be shared.
Permissions boundary	Your account	Managed policy used as a maximum	Delegate identity creation without allowing privilege beyond an approved ceiling.

Boundary example

Developer policy
Allows EC2 + S3 + IAM actions

∩

Boundary
Allows only EC2 + S3

=

Effective maximum
EC2 + S3; IAM is blocked

A boundary does not grant EC2 or S3. It only says those permissions may be granted by identity policies. The developer still needs an identity policy that allows the required actions.

Reusable policy design

Good pattern

- Customer managed policy per job capability.
- Version controlled as infrastructure as code.
- Validated before deployment.
- Refined using access activity.

Risky pattern

- Many copied inline policies.
- Wildcards added to fix errors quickly.
- No owner or review date.
- Broad AWS policy kept forever.

Two different meanings of "version"

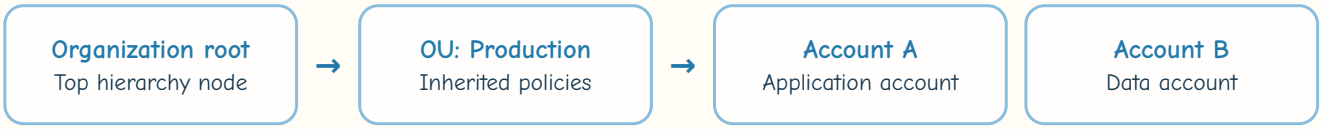
JSON **`Version` element:** policy language syntax version, normally `2012-10-17`.

Managed policy versions: saved revisions of a customer managed policy, with one selected as default.

Practical rule: Start with a managed policy only to learn the access pattern, then move toward a customer managed policy that contains only observed and approved permissions.

Organization guardrails: SCPs and RCPs

AWS Organizations can limit principals in member accounts and protect resources in member accounts.



Service Control Policy - SCP

Sets the maximum available permissions for IAM principals in affected **member accounts**.

- Does not grant access.
- Applied at root, OU or account.
- Inherited down the hierarchy.
- Can affect member-account root users, subject to documented exceptions.

Resource Control Policy - RCP

Sets the maximum permissions available when principals access supported resources in affected **member accounts**.

- Does not grant access.
- Helps centrally restrict external access to organization resources.
- Applies regardless of whether the calling principal is inside or outside the organization.

Quick comparison

Question	SCP	RCP
What is guarded?	Principals managed by member accounts	Supported resources in member accounts
Does it grant?	No	No
Main perspective	"What may identities in this account do?"	"What access may this resource receive?"
Management account	Does not restrict its users/roles	Does not protect its resources

Rollout method

Test a new guardrail in a sandbox OU, monitor denied actions, move a small number of accounts, then expand gradually. A broad deny deployed at the organization root can disrupt critical services.

Important exception awareness: Service-linked roles and some documented tasks are not restricted in the same way. Always verify the current service and Organizations documentation before relying on a guardrail.

Resource policies and cross-account access

A resource can trust a principal directly, or a target account can expose a role that callers assume.

Resources with policy controls

Examples include S3 buckets, KMS keys, SQS queues, SNS topics, Lambda functions, Secrets Manager secrets and IAM role trust policies.

Principal is used here to name the trusted caller.

Avoid broad principals

`"Principal": "*"` with an Allow can create public or broadly accessible resources. Narrow the principal and add conditions such as organization, source ARN or external ID.

Pattern A - direct resource access

Account A role
Identity policy allows target action
→
Account B resource
Resource policy trusts Account A role
→
Resource API
Both sides evaluated

Pattern B - assume a role in the target account

Account A principal
Allowed to call `sts:AssumeRole`
→
Account B role
Trust policy accepts caller
→
STS session in B
Uses role permissions in B
→
Account B resources
Authorized as assumed role

Trust policy skeleton

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::111122223333:role/BuildRole" },
  "Action": "sts:AssumeRole",
  "Condition": { "StringEquals": { "sts:ExternalId": "agreed-value" } }
}
```

External ID

Helps protect third-party role access from the confused-deputy problem.

PrincipalOrgID

Can restrict supported policies to principals belonging to your AWS Organization.

KMS reminder

Cross-account encrypted data also needs compatible KMS key permissions.

Preferred mental model: Cross-account trust is a handshake. Caller permission and target trust/resource permission must align.

AWS IAM Identity Center

The recommended central access layer for workforce users across multiple AWS accounts.

Simple definition: IAM Identity Center connects or stores workforce identities, assigns users and groups to AWS accounts and applications, and gives them temporary access through permission sets.

How account access works

Identity source External IdP or Identity Center directory → **User / group** Workforce identity + **Permission set** Policies + session settings + **AWS account** Assignment target → **IAM role** Temporary session credentials

Permission set

- Defines a level of account access.
- Can include AWS managed, customer managed and inline policy content.
- Has a configurable session duration.
- Is provisioned as an IAM role in assigned accounts.

AWS access portal

- User signs in once.
- Sees assigned accounts and roles.
- Chooses the least-privileged suitable permission set.
- Gets console or CLI temporary access.

IAM user versus Identity Center user

Area	IAM user	Identity Center workforce user
Scope	One AWS account	Central assignments across many accounts/apps
Credentials	Can have long-term password/keys	Federated sign-in and temporary role sessions
Group location	IAM user groups in one account	Identity Center or connected external IdP
Preferred workforce model	No, except documented cases	***Yes***

CLI sign-in

```
aws configure sso
aws sso login --profile production-readonly
aws sts get-caller-identity --profile production-readonly
```

Remember: Identity Center is not a replacement for all IAM roles. It uses IAM roles in accounts to deliver the assigned permissions.

Federation: SAML, OIDC and customer identity

Federation lets an external identity obtain temporary AWS access without creating a long-term IAM user.

EXTERNAL IDENTITY → ASSERTION / TOKEN → AWS STS → IAM ROLE SESSION

SAML 2.0

Common for enterprise workforce federation from a corporate identity provider into AWS roles or through Identity Center.

OpenID Connect - OIDC

Common for web identity, CI/CD and supported workload federation using signed identity tokens and claims.

IAM Identity Center

Preferred central workforce experience for organizations with multiple AWS accounts and applications.

Which service for which population?

Population	Recommended starting point	Why
Employees and administrators	Corporate IdP + IAM Identity Center	Central SSO, groups, account assignments and temporary credentials
External CI/CD platform	OIDC provider + trusted IAM role	No stored AWS access key; short-lived deployment session
AWS-hosted workload	Native IAM role for the compute service	AWS supplies and refreshes temporary credentials
Customers of your application	Amazon Cognito or another application identity solution	Customers should not be created as IAM workforce users

Federation flow

01 User or workload authenticates to the trusted identity provider.

02 The provider issues a signed SAML assertion or OIDC token containing identity claims.

03 AWS verifies the provider, token audience, signature and trust-policy conditions.

04 STS returns temporary role credentials or an AWS console session.

05 CloudTrail records activity under the resulting federated or assumed-role principal.

Trust policy warning: Do not trust every token from an OIDC provider. Restrict audience, repository, branch, subject, organization or other reliable claims for the exact use case.

Benefit: Password policy, MFA and joiner/mover/leaver lifecycle stay in the workforce identity system while AWS receives time-limited role sessions.

Roles for AWS workloads and services

Give code a role that matches where it runs; do not copy access keys into the workload.

EC2

Instance role

IAM role is delivered to EC2 through an instance profile.

LMB

Lambda execution role

Lambda assumes the role when running your function.

ECS

Task role

Application containers receive task-specific credentials.

EKS

Pod identity role

Pods obtain scoped AWS permissions through supported EKS identity mechanisms.

SVC

Service role

An AWS service assumes a role to perform actions on your behalf.

SLR

Service-linked role

Predefined role linked to and controlled by a particular AWS service.

EC2 instance profile

IAM role
Trusts EC2 + has permissions

→

Instance profile
Container used to pass the role

→

EC2 instance
Metadata exposes temporary credentials

→

AWS SDK
Retrieves and refreshes automatically

Service role

You normally control its trust and permissions for a service use case, such as an EC2 application role or a deployment role used by CloudFormation.

Service-linked role

The AWS service defines the role and required permissions. You can inspect it, but the service controls much of its lifecycle and policy content.

Outside AWS

For workloads running in a data center, another cloud or a supported edge environment, evaluate federation methods such as **IAM Roles Anywhere**, OIDC, EKS hybrid patterns or other service-specific temporary-credential mechanisms.

Common mistake: Reusing one powerful role for every application on a host or cluster. Separate workload identities so CloudTrail, least privilege and incident response remain meaningful.

Rule: Human gets federation. AWS workload gets a service-native role. External workload gets a federation mechanism. Long-term keys are the exception.

Practical flow: EC2 application to S3 and KMS

Authentication, authorization, encryption and network reachability are separate layers.

EC2 application No key in code → **Instance metadata** Temporary role credentials → **AWS SDK** Signs S3 request → **S3 bucket** IAM + bucket policy → **KMS key** If SSE-KMS is used

Required pieces

1. Role trust

The role trust policy allows the EC2 service principal to assume the role.

2. Instance profile

Passes the IAM role to the EC2 instance. The console usually creates it with the EC2 role.

3. S3 permissions

The role allows only required bucket/object actions on the exact bucket or prefix.

4. KMS permissions

For SSE-KMS, the IAM policy and KMS key authorization must permit the required cryptographic action.

5. Resource controls

Bucket policy, SCP, RCP, boundary, endpoint policy and explicit denies must not block the request.

6. Network path

DNS, routes, proxy, internet/NAT path or S3 VPC endpoint must permit reachability.

Minimal role permission idea

```
Allow: s3:ListBucket on arn:aws:s3:::app-data
       Condition: s3:prefix = inbound/*
Allow: s3:GetObject, s3:PutObject
       on arn:aws:s3:::app-data/inbound/*
Allow: kms:Decrypt, kms:GenerateDataKey
       on the approved customer-managed KMS key, when required
```

When S3 returns 403

Check	Example problem
Active principal	Application is using an old environment access key instead of the instance role.
Object/bucket ARN	Policy grants bucket ARN for an object action or misses the prefix.
Explicit deny	Bucket policy requires a particular VPC endpoint or TLS.
KMS	S3 access is allowed but the KMS key denies decrypt or data-key use.

Operational check: Confirm the caller identity first, then IAM/resource policies, then KMS, then the network path.

IAM in a three-tier AWS application

IAM controls access to AWS APIs; the database engine controls schemas, rows and SQL permissions.

Presentation tier
Browser / mobile / UI

→

Application tier
EC2 / ECS / Lambda role

→

Data tier
RDS + S3 + KMS + queues

User identity

The application user might authenticate through Amazon Cognito, a corporate identity provider or the application's own supported identity service.

Database identity

RDS or the database engine uses database users/roles, passwords, certificates, IAM database authentication where supported, and SQL grants.

Workload identity

The application process uses an IAM role to call AWS APIs such as S3, Secrets Manager, SQS, DynamoDB or KMS.

Storage identity

S3 access is controlled by IAM/resource policies; object encryption may additionally require KMS authorization.

Saving a cheque-processing transaction

01 User submits amount, account details and cheque images to the application.

02 Application validates the request and authorizes the business operation.

03 Structured transaction data is inserted into RDS using database credentials or supported IAM DB authentication.

04 Large images are uploaded to S3 using the application's IAM role.

05 The database stores the S3 object key and business metadata, not a long-lived AWS credential.

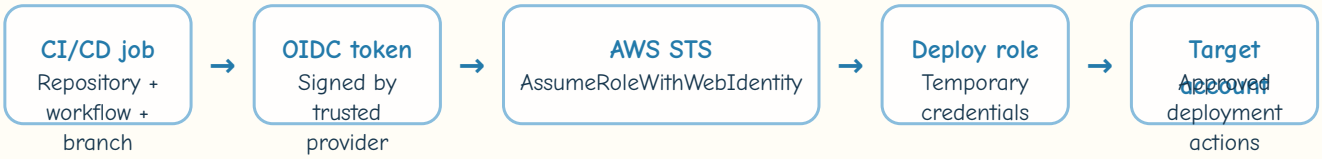
IAM decides: May the app call `s3:PutObject`, `secretsmanager:GetSecretValue` or `rds-db:connect`?

Database decides: May this database identity insert into a table, update a row or execute a stored procedure?

Key relationship: The application role gets AWS service access. It does not automatically give permission to read every database row or to bypass application authorization.

DevOps identity: CI/CD with OIDC

A deployment platform can exchange its signed identity token for a short-lived AWS role session.



Trust policy idea

```
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.example-ci.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": { "token.example-ci.com:aud": "sts.amazonaws.com" },
    "StringLike":   { "token.example-ci.com:sub": "repo:company/payments:ref:refs/heads/main" }
  }
}
```

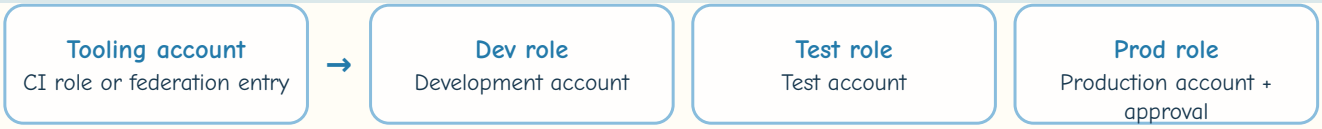
Security advantages

- No AWS secret stored in the CI platform.
- Credentials expire automatically.
- Trust can be restricted by token claims.
- Each job has a traceable role session.

Required controls

- Exact provider and audience.
- Repository, branch/environment or workflow restrictions.
- Least-privilege deployment role.
- Protected production approval path.

Multi-account deployment pattern



Avoid: one administrator role trusted by every repository and branch. Split roles by environment and workload, and constrain both trust and permissions.

Audit: Give each job a useful role-session name so CloudTrail can link the API activity to the repository, run or deployment.

ABAC, policy conditions and iam:PassRole

Conditions make authorization context-aware; PassRole controls which roles a person may attach to a service.

RBAC

Role-based access control grants permissions based on job role or capability.

Example: DatabaseOperators may reboot selected RDS instances.

ABAC

Attribute-based access control grants access when principal, session and resource attributes match.

In AWS, attributes are commonly represented by tags.

Useful condition ideas

Condition context	Example use
`aws:PrincipalTag/Project` and `aws:ResourceTag/Project`	Allow project members to manage resources tagged for their project.
`aws:MultiFactorAuthPresent`	Require MFA for selected IAM user/API scenarios.
`aws:RequestedRegion`	Restrict applicable service operations to approved Regions.
`aws:SourceVpce`	Require access through an approved VPC endpoint where supported.
`aws:PrincipalOrgID`	Limit supported resource access to principals from your organization.
`aws:RequestTag` / `aws:TagKeys`	Require approved tags when resources are created or changed.

What is iam:PassRole?

Engineer Creates Lambda / EC2 / ECS resource → **Passes approved role** `iam:PassRole` → **AWS service** Later assumes that role → **Workload actions** Role permissions

PassRole is not AssumeRole. The engineer is authorizing a service to use a role; the engineer does not receive the role session.

Escalation risk: Someone who can create a compute resource and pass a powerful role may indirectly gain powerful access.

PassRole safeguards

- [ ] Scope `iam:PassRole` to approved role ARNs, not every role.
- [ ] Use role naming paths and delegated boundaries to separate application roles.
- [ ] Use `iam:PassedToService` where appropriate to restrict the destination service.
- [ ] Review both the creator's service permissions and the permissions of roles they may pass.

ABAC benefit: A stable policy can support many projects when reliable tags are governed. Without tag governance, ABAC can become unpredictable or exploitable.

Audit, analysis and least-privilege refinement

Use evidence from logs and analysis tools to understand access and safely reduce permissions.

AWS CloudTrail

Records AWS API activity, principal identity, event time, source, request parameters and response details.

IAM Access Analyzer

Finds external/public access, validates policies, supports access previews and can help generate/ refine policies.

Credential report

Account-wide CSV showing IAM user password, access-key and MFA status for audit purposes.

Last accessed data

Shows when services or actions were last used to help remove permissions that appear unnecessary.

Policy Simulator

Tests whether selected identity policies would allow or deny actions under supplied context.

CloudWatch / EventBridge

Can alert or automate response for sensitive IAM, root, policy and role events from logs.

Access Analyzer capabilities

Before deployment

- Validate JSON and security findings.
- Check whether a changed policy introduces new access.
- Preview supported external access changes.

After deployment

- Detect supported public or cross-account access.
- Review unused access where configured.
- Generate policy ideas from CloudTrail activity.

Operating cadence

Frequency	Suggested review
Continuous	Alerts for root use, policy changes, access-key creation, suspicious AssumeRole and public resource changes.
Weekly	New Access Analyzer findings, failed access trends and high-privilege role activity.
Monthly	Credential report, unused users/keys, stale roles, direct user policies and inactive permission sets.
Quarterly	Access reviews, SCP/RCP testing, administrator paths, break-glass drill and policy ownership.

Least-privilege refinement loop

Observe CloudTrail + last accessed → **Draft** Smaller customer policy → **Validate** Access Analyzer → **Test** Allow + deny scenarios → **Deploy** Monitor and roll back

Evidence rule: "Not used recently" is a signal, not proof. Confirm application schedules, disaster-recovery paths and rare operational tasks before removing access.

Troubleshooting AccessDenied and role failures

Follow the request from active credentials to every policy layer instead of adding wildcards.

First identify the active caller

```
aws sts get-caller-identity
# Check Account, Arn and UserId. Confirm profile, environment variables,
# container/instance role and any assumed-role session.
```

Common symptoms

Symptom	Likely area	What to check
`ExpiredToken` / invalid token	Authentication	STS expiration, session token, local clock, stale cached SSO session.
`AccessDenied`	Authorization	Missing allow, wrong ARN, condition mismatch or explicit deny.
Cannot assume role	Trust handshake	Caller has `sts:AssumeRole`; role trusts exact principal; external ID/MFA/claims match.
Cannot create service resource with role	PassRole	Caller has `iam:PassRole` on the role and service is trusted.
S3 allowed but encrypted object fails	KMS	KMS key policy/grant and IAM permissions for decrypt/data key.
Policy looks correct but still fails	Guardrail/resource path	Boundary, session policy, SCP, RCP, endpoint policy, bucket/key policy.

Systematic checklist

1. Copy the exact API action, resource ARN, Region and error.
2. Confirm the active account and principal ARN.
3. Find the CloudTrail event and request context.
4. Check identity policy for an applicable Allow.
5. Check trust or resource policy if another account/ resource is involved.
1. Check boundary, session policy, SCP, RCP and endpoint policy.
2. Search every layer for explicit Deny and condition mismatches.
3. Check KMS authorization for encrypted data.
4. Check network reachability separately.
5. Change the smallest policy, retest, then remove temporary diagnostics.

Frequent ARN mistakes

Bucket vs object

`arn:aws:s3:::bucket` differs from `arn:aws:s3:::bucket/*`.

Role vs session

Trust/resources may refer to IAM role ARN while logs show an STS assumed-role ARN.

Wrong account

A same-named role in another account is a different principal.

Do not fix by habit: Adding `Action: "*"` or `Resource: "*"` may hide the real issue and create a security incident.

Best diagnostic question: Which exact policy layer made this exact request ineffective?

AWS IAM security and change checklist

A practical pre-production review for identities, roles and policy changes.

Root and workforce

- [ ] Root has MFA, no access keys and monitored recovery channels.
- [ ] Humans use federation and temporary credentials where possible.
- [ ] Administrator access is separate from everyday read/change access.
- [ ] Inactive users, permission sets and assignments are reviewed.
- [ ] Break-glass access has approval, logging and a test procedure.

Workloads and secrets

- [ ] Each workload has a dedicated, traceable role.
- [ ] No access keys are embedded in code, images or pipelines.
- [ ] Trust policy is restricted to the required service/provider/claims.
- [ ] Role permissions are scoped to exact actions and resources.
- [ ] Secrets are stored in an approved secret-management service.

Policies and guardrails

- [ ] Policy has an owner, purpose, review date and change record.
- [ ] Access Analyzer validation findings are resolved or documented.
- [ ] Wildcards are justified and conditions are tested.
- [ ] Boundary, SCP, RCP, session and resource policies were considered.
- [ ] PassRole is limited to approved role ARNs and services.

Audit and resilience

- [ ] CloudTrail coverage and log protection are in place.
- [ ] Alerts exist for root use and sensitive IAM changes.
- [ ] Role-session names/source identity support attribution.
- [ ] Rollback path is tested before a broad guardrail deployment.
- [ ] Emergency and DR paths still work with proposed restrictions.

Before implementing a policy change

Business need What exact task? → **Design** Action, resource, condition → **Validate** Syntax + Analyzer → **Test** Positive + negative → **Deploy** Small scope + monitor

Change record should capture

Principal, target account, exact policy ARN/version, old and new effective access, test evidence, expected denied actions, approver, implementation time and rollback steps.

Least privilege is a lifecycle, not a one-time policy. Start safely, observe real usage, refine access and repeat.

Hands-on AWS IAM learning labs

Use a non-production account and clean up every resource after testing.

Lab 1 - Identity Center session

1. Create or use a sandbox permission set with read-only access.
2. Assign a test user/group to the sandbox account.
3. Configure an AWS CLI SSO profile.
4. Run `aws sso login` and `aws sts get-caller-identity`.
5. Try one allowed and one denied action.

Lab 2 - EC2 role to S3

1. Create a private test bucket and prefix.
2. Create an EC2 role with exact list/get/put permissions.
3. Attach it through an instance profile.
4. Use the AWS CLI from EC2 without configuring keys.
5. Remove one action and observe the denial.

Lab 3 - Policy anatomy

1. Write a customer managed policy for one S3 prefix.
2. Validate it with IAM Access Analyzer.
3. Use Policy Simulator for list/get/delete.
4. Test resource ARN differences.
5. Record the effective result.

Lab 4 - AssumeRole

1. Create a sandbox role with a narrow trust policy.
2. Grant a test principal `sts:AssumeRole`.
3. Assume the role with AWS CLI.
4. Export/use the temporary session safely.
5. Inspect the CloudTrail event and session ARN.

Useful inspection commands

```
aws sts get-caller-identity
aws iam get-role --role-name AppRole
aws iam list-attached-role-policies --role-name AppRole
aws iam list-role-policies --role-name AppRole
aws iam get-policy-version --policy-arn POLICY_ARN --version-id v1
aws iam generate-credential-report
aws iam get-credential-report
```

Lab 5 - Audit and refine

1. Generate a credential report and identify unused/unsafe IAM user credentials.
2. Create an Access Analyzer in the appropriate zone of trust.
3. Review findings for a sandbox resource policy.
4. Use last-accessed information before reducing a role policy.
5. Document why each retained permission is required.

Safety: Do not run broad deny tests, root-user experiments or SCP/RCP labs in production. Organization guardrails can interrupt every account beneath the attachment point.

Learning evidence: Save the policy, caller ARN, command, expected result, actual result and CloudTrail event ID for each lab.

AWS IAM one-page cheat sheet

The minimum set of relationships to remember during design and troubleshooting.

Core definitions

Principal	Caller making an AWS request.
User	Account-local identity with possible long-term credentials.
Group	Permission collection for IAM users; not a principal.
Role	Assumable identity that produces temporary credentials.
Policy	JSON permission or guardrail document.
Trust policy	Who/what may assume a role.
STS	Issues temporary security credentials.
ARN	Exact identifier for an AWS resource or principal.

Decision formulas

```
DEFAULT DENY
+ APPLICABLE ALLOW
- ANY EXPLICIT DENY
= FINAL DECISION
```

```
ROLE SESSION = TRUST CHECK
+ STS TEMPORARY CREDENTIALS
+ ROLE PERMISSIONS
```

```
EFFECTIVE ACCESS = GRANTS
∩ BOUNDARY ∩ SESSION
∩ SCP/RCP
THEN EXPLICIT DENY WINS
```

Which identity should I use?

Need	Starting choice
Employee access to many accounts	External IdP + IAM Identity Center + permission sets
Application running on EC2	EC2 role through an instance profile
Lambda calling S3/DynamoDB	Lambda execution role
ECS application	ECS task role
CI/CD outside AWS	OIDC federation to a deployment role
Access another AWS account	Assume a narrowly trusted role in the target account
Application customers	Cognito or another customer identity solution, not IAM users

Fast troubleshooting order

Caller → Action / ARN → Identity allow → Trust / resource → Guardrails → KMS / network

Golden rules: Prefer temporary credentials. Protect root. Grant least privilege. Avoid broad trust. Log every session. Test intended denies.

AWS IAM compared with Microsoft Entra ID

The concepts overlap, but the products and security boundaries are not one-to-one equivalents.

Microsoft concept	Closest AWS concept	Important difference
Microsoft Entra tenant	AWS Organization plus its accounts	An AWS account is also a strong resource and billing boundary; the mapping is not exact.
Entra user	IAM Identity Center user or IAM user	Identity Center is preferred for workforce; IAM user exists inside one account.
Entra group	Identity Center group or IAM user group	IAM user groups organize only IAM users and are not principals.
Azure RBAC role assignment	IAM policy/permission set applied to a principal	AWS combines several policy types rather than one direct role-assignment model.
Service principal	IAM role / federated principal	AWS workload access usually centers on assumable roles and STS sessions.
Managed identity	Service-native IAM role	EC2/Lambda/ECS receive automatically managed temporary role credentials.
Conditional Access	IdP/Identity Center controls + IAM conditions + org guardrails	AWS IAM has no single direct equivalent to Entra Conditional Access.
Azure subscription	AWS account	Both contain resources, but governance hierarchy and identity integration differ.
Azure management group	AWS Organization root / OU	AWS Organizations uses accounts/OUs and SCP/RCP guardrails.
PIM eligible role	Temporary elevated access solution + short role session	AWS implementation depends on Identity Center, workflows and third-party/self-managed elevation patterns.

Architecture translation

Azure mental model

User/managed identity authenticates to Entra ID, obtains a token, then Azure RBAC and the resource decide access.

AWS mental model

Principal uses credentials or assumes a role through STS, then AWS evaluates identity, resource and guardrail policies.

```
ENTRA: IDENTITY → TOKEN → RBAC / RESOURCE
AWS: PRINCIPAL → STS SESSION → MULTI-POLICY EVALUATION
```

Cross-cloud memory shortcuts

People

Entra workforce users ↔ Identity Center workforce users.

Applications

Managed identity/service principal ↔ service role/federated role.

Resources

Azure RBAC scope ↔ AWS action/resource/condition policies.

Do not translate names literally. Start from the security question: who authenticates, which temporary credential is issued, where permissions are defined, and which guardrails apply?

Glossary and official learning references

Use the current AWS documentation for service-specific actions, quotas and newly released controls.

Abbreviations

IAM	Identity and Access Management
STS	Security Token Service
MFA	Multi-Factor Authentication
ARN	Amazon Resource Name
IdP	Identity Provider
SAML	Security Assertion Markup Language
OIDC	OpenID Connect
RBAC	Role-Based Access Control
ABAC	Attribute-Based Access Control
SCP	Service Control Policy
RCP	Resource Control Policy

Official AWS references

- AWS Identity and Access Management documentation
- Security best practices in IAM
- Root user best practices
- IAM identities
- IAM roles
- Temporary security credentials
- Policies and permissions in IAM
- Policy evaluation logic
- What is IAM Identity Center?
- AWS Organizations SCPs
- AWS Organizations RCPs
- IAM Access Analyzer

Final memory map

Principal Human, workload, service → **Credential** Prefer temporary → **Request** Action + resource + context → **Policies** Grants + limits + denies → **Decision** Allow / deny + audit

PROTECT ROOT | FEDERATE PEOPLE | ROLE-ENABLE WORKLOADS
GRANT LEAST PRIVILEGE | EXPLICIT DENY WINS | LOG AND REVIEW

Study approach

Revisit pages 4, 9, 13, 17, 20, 25 and 28 after each hands-on lab. IAM becomes easier when every error is translated into: principal, credential, action, resource, context and applicable policy layers.

Prepared as a learning aid using official AWS documentation available in August 2026. AWS services and recommendations evolve; verify current service documentation before production implementation.